接口与 MJPEG 格式。 表 7.22 是服务器提供的路径。

表 7.22: camserver 的 HTTP 接口

路径	内容
/	HTML 页面，内嵌 <img src="/stream">
/stream	MJPEG 流，multipart/x-mixed-replace; boundary=frame
/snapshot.jpg	单帧 JPEG：正在推流时给最新一帧，否则临时上电拍一帧再断电
/status	纯文本：摄像头状态、尺寸、质量、观看人数、帧数、帧率、最近一帧大小
其他	404

MJPEG over HTTP 没有专门的协议，只是利用了 multipart/x-mixed-replace 这种内容类型：响应永远不结束，每一部分是一整幅 JPEG，浏览器收到新的一部分就替换掉上一幅。线上的字节流是这样的：

```
HTTP/1.1 200 OK
Content-Type: multipart/x-mixed-replace; boundary=frame
Cache-Control: no-cache, no-store, must-revalidate
Connection: close

--frame
Content-Type: image/jpeg
Content-Length: 18143

<18143 字节：FF D8 ... FF D9>
--frame
Content-Type: image/jpeg
Content-Length: 18144
...
```

每一部分都带 Content-Length，客户端不需要在 JPEG 数据里搜索边界串。

测试。 QEMU 的 raspi3b 没有摄像头，它的 USB 网卡在当前版本下也没有被 dwc2 驱动接受，无法端到端测试。因此先在宿主机上测试服务器逻辑：把 socket_listen、socket_accept、epoll_* 和 /dev/video0 换成 POSIX 的实现（假摄像头每 100 ms 产生一帧带移动红条的 640×480 RAW10），同一份 camserver.c 不加修改地编译运行：

```
/status            200, camera: idle
/                  200 text/html
/nope              404
静默预连接          2.0 s 后被丢弃，同时到来的 /status 正常返回
/snapshot.jpg      200 image/jpeg，解码为 640x480
5 个并发 /stream   前 4 个各收到 26~28 帧，每帧都能解码；第 5 个 503 too many viewers
观看者全部离开      camserver: no viewers, camera stopped
```

真机上这样运行：

```
$ wifi                 # 读 /etc/wifi.conf，关联并 DHCP，记下 wlan0 的 IP
$ camserver -s         # 先用 320x240；-q 调质量，-n 限制观看人数
camserver: listening on port 8080 (320x240, quality 75, max 4 viewers)
```

然后在同一局域网里用浏览器打开 http://<IP>:8080/。串口每 3 秒打印一行 camserver: N.N fps, B bytes/frame, K viewer(s)。帧率受三件事限制：等待下一帧的到来、用户态整数去马赛克和 JPEG 编码，以及 Wi-Fi 上的 TCP 吞吐；半尺寸 -s 能同时减轻后两项。